

第三章 2 英文与中文字符的二进制表示 (Binary Representation of Characters)

秦磊华 汪承宁



3.5 非数值数据

计算机内如何使用二进制表示字符数据。

1. ASCII 码

位 数				w7	0	0	0	0	1	1	1	1
				w6	0	0	1	1	0	0	1	1
				w5	0	1	0	1	0	1	0	1
w4	w3	w2	w1	行	0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	`	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

1) **ASCII**: American Standard Code for Information Interchange (ANSI 7bits)

2) 7bits, MSB=0, 共可表示128个字符

- ◆ 52 Letters : a-z, A-Z
- ◆ 10 Digits: 0-9
- ◆ 34 Symbols :! @ # \$ % ^ & * () ...
- ◆ 32 Control characters : <CR> <BEL> <ESC> <LF>

EBCDIC-Extended Binary-Coded Decimal Interchange Code (IBM 8bits)

Unicode: Universal Multiple-Octet Coded Character Set“，简称为UCS（统一码、万国码、单一码）



3.5 非数值数据

Unicode:

Unicode通常用两个字节表示一个字符，原有的英文编码从单字节变成双字节，只需要把高字节全部填为0就可以。

以满足跨语言、跨平台进行文本转换、处理的需求。1990年开始研发，1994年正式公布。

2. 汉字编码 用于中文信息处理

1) GB2312简体中文编码表

- ◆ GB2312标准共收录6763个汉字(一级, 二级)及682个全角字符拉丁字母、希腊字母、日文平假名及片假名字母、俄语字母等;
- ◆ 整个字符集分成**94个区**, **每区有94个位**;
 - 01-09区为特殊符号
 - 10-15区没有编码
 - 16-55区为一级汉字, 按拼音排序, 共3755个
 - 56-87区为二级汉字, 按部首 / 笔画排序, 共3008个
 - 88-94区没有编码



3.5 非数值数据

2. 汉字编 码

GB2312汉字编码字符集对照表

第01区	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
A1A0			、	。	．	～	ˇ	¨	＃	々	〒	～		…	’	’
A1B0	“ ”	” ”	[]	〈 〉	《 》	「 」	『 』	【 】	〒	〒	〒	〒	〒	〒	〒	〒
A1C0	±	×	÷	:	^	√	Σ	Π	U	∩	∈	::	√	⊥		∠
A1D0	∩	○	∫	φ	≡	≡	≈	≈	∞	≠	≠	≠	≤	≥	∞	∴
A1E0	∴	♂	♀	°	’	”	℃	\$	¤	¢	£	‰	§	No	☆	★
A1F0	◦	●	®	◇	◆	□	■	△	▲	※	→	←	↑	↓	＝	
第02区	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
A2A0																
A2B0		1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.	13.	14.	15.
A2C0	16.	17.	18.	19.	20.	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)
A2D0	(12)	(13)	(14)	(15)	(16)	(17)	(18)	(19)	(20)	①	②	③	④	⑤	⑥	⑦
A2E0	⑧	⑨	⑩			(一)	(二)	(三)	(四)	(五)	(六)	(七)	(八)	(九)	(十)	
A2F0		I	II	III	IV	V	VI	VII	VIII	IX	X	XI	XII			
第03区	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F



3.5 非数值数据

2. 汉字编 码

GB2312汉字编码字符集对照表

A9F0																
第10区	AAA0-AAFF	没有编码														
第11区	ABA0-ABFF	没有编码														
第12区	ACA0-ACFF	没有编码														
第13区	ADA0-ADFF	没有编码														
第14区	AEA0-AEFF	没有编码														
第15区	AFA0-AFFF	没有编码														
第16区	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
B0A0		啊	阿	埃	挨	哎	唉	哀	皑	痍	蔼	矮	艾	碍	爱	隘
B0B0	鞍	氨	安	俺	按	暗	岸	胺	案	肮	昂	盎	凹	敖	熬	翱
B0C0	袄	傲	奥	懊	澳	芭	捌	扒	叭	吧	笆	八	疤	巴	拔	跋
B0D0	靶	把	耙	坝	霸	罢	爸	白	柏	百	摆	佰	败	拜	稗	斑
B0E0	班	搬	扳	股	颁	板	版	扮	拌	伴	瓣	半	办	绊	邦	帮
B0F0	梆	榜	膀	绑	棒	磅	蚌	镑	傍	谤	苞	胞	包	褒	剥	
第17区	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
B1A0		薄	雹	保	堡	饱	宝	抱	报	暴	豹	鲍	爆	杯	碑	悲

区位码 机内码

3.5 非数值数据

3. 汉字编码标准发展

- ◆ GB2312-1980(GB0)(简体)

6763个汉字

- ◆ GB13000-1993

20902个汉字 (Unicode 1.1版本)

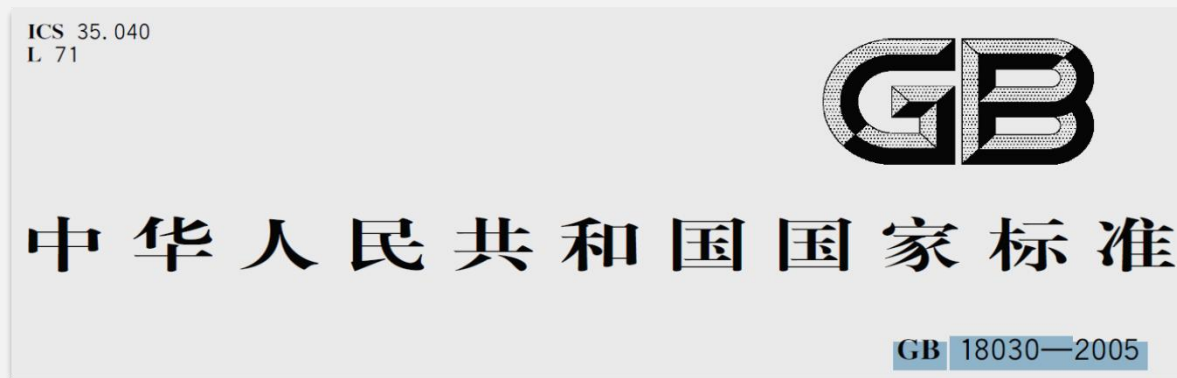
- ◆ 汉字扩展规范GBK1.0 标准1995 (非国家标准) : 21003个字符 (兼容GB2312)

- ◆ Big5-2003

大五码,13,060个汉字, 使用繁体中文社群中最常用的汉字字符集标准

- ◆ GB18030-2005 (1/2/4字节编码: 可变长编码)

70,244个汉字, 对GB 2312-1980完全向后兼容, 与GBK基本向后兼容, 并支持Unicode (GB 13000) 的所有码位。



3.5 非数值数据

3. 汉字编码标准发展

■ `<META content="text/html; charset=gb2312"`

... `http-equiv=Content-Type>`

■ `charset=gb2312` 简体中文

`charset=big5` 繁体中文

`charset=EUC_KR` 韩语

`charset=Shift_JIS` 或 `EUC_JP` 日语

`charset=KOI8-R/Windows-1251` 俄语

`charset=iso-8859-2` 中欧语系

`charset=utf-8 unicode` 多语言

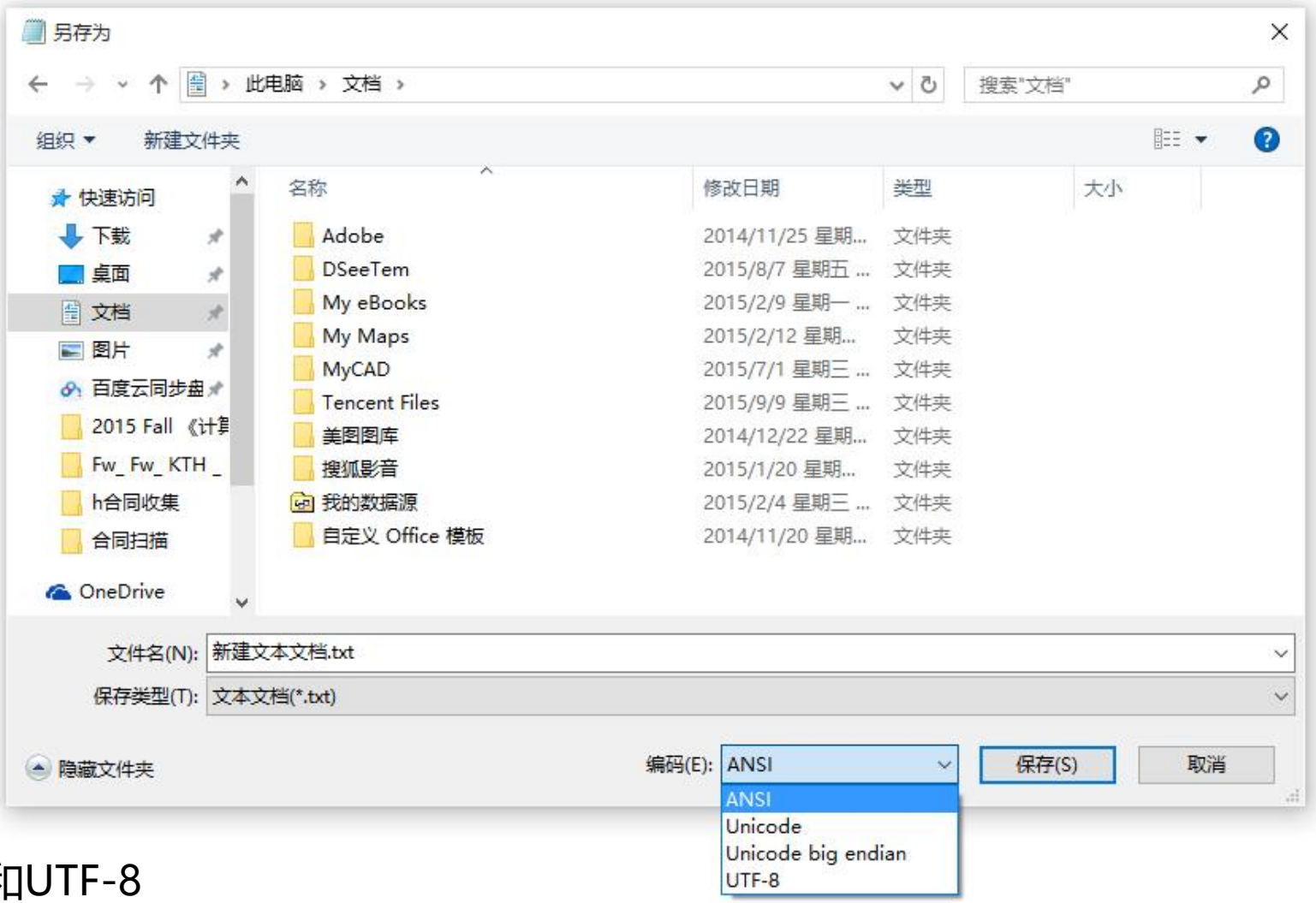
1. 打开电脑，点击主菜单，找到并点击“控制面板”选项，之后找到并点击“时钟、语言和区域”选项。
2. 在打开的页面中找到并点击“语言”选项。
3. 点击并打开“添加语言”选项，即可找寻想要添加的语言包。也可以点击右上的搜索框，输入想添加的语言包名称。
4. 输入想添加的语言包并搜索之后，点击该语言包并点击下方的“添加”按钮，语言包便安装...



3.5 非数值数据

3. 汉字编码标准发展

记事本编码区别？

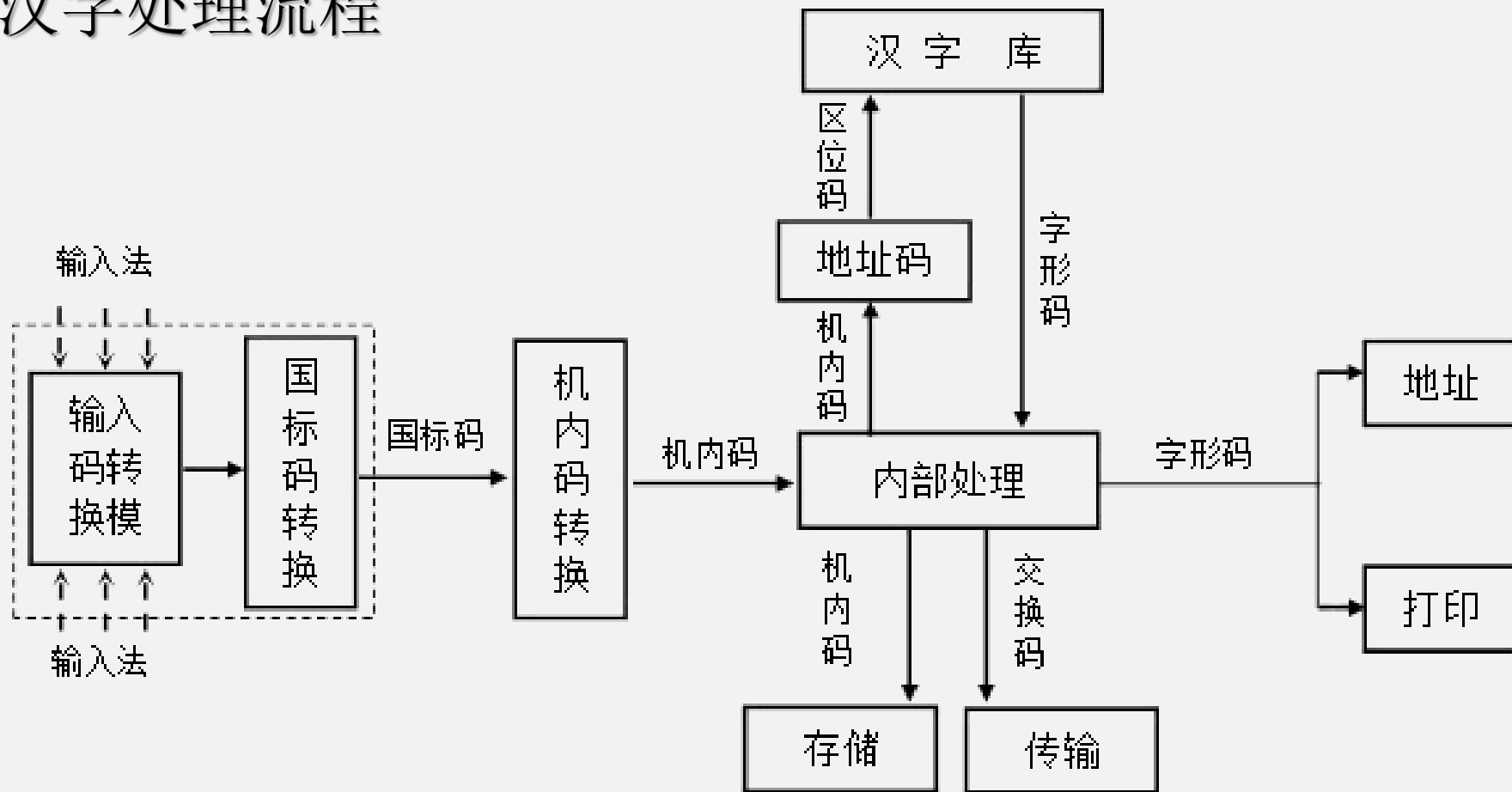


字符编码笔记：ASCII，Unicode和UTF-8

http://www.ruanyifeng.com/blog/2007/10/ascii_unicode_and_utf-8.html

3.5 非数值数据

4. 汉字处理流程



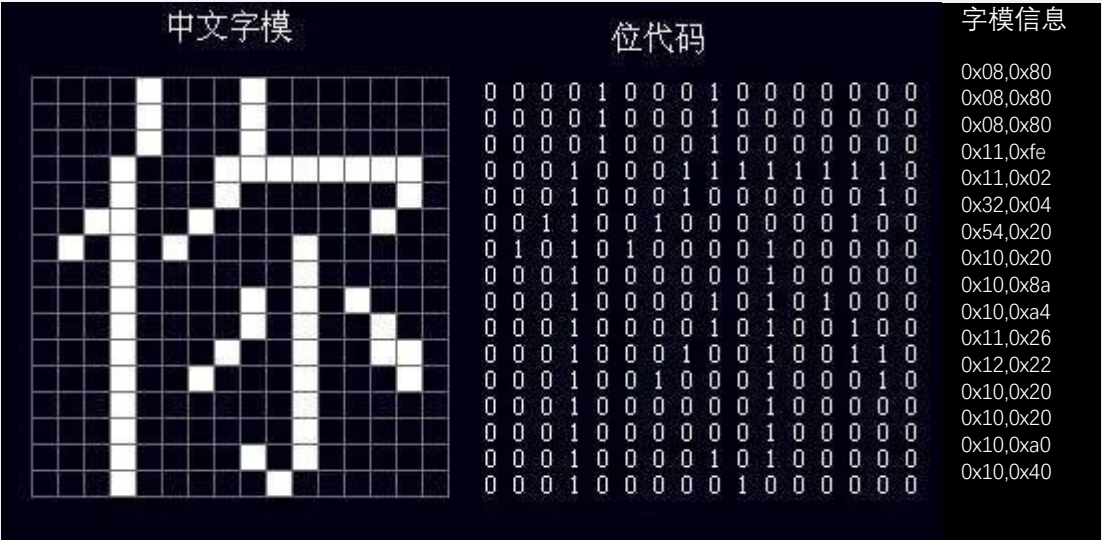
4. 汉字处理流程

区位码 + 2020H + 8080H 变成机内码的原因

- ◆ GB2312依然保留26个英文字母和一些特殊符号的编码，**覆盖ASCII中的相同部分**，同时将ASCII中前32个控制字符继续沿用(即十六进制**20H**)。因此，区位码要加上2020H才能得到国标码。
- ◆ 汉字国标码中用2个字节表示一个汉字，且每个字节的高位为0(见上一条以及一、二级汉字总数6763， $128 \times 128 = 16384$)
- ◆ 上述**覆盖而非兼容ASCII码**的设计在应用中出现了乱码。原因是**两字节表示的1个汉字与2个ASCII字符无法区分**。为此，在国标码的基础上，将2个字节的最高位均**设置为1**，因为ASCII中使用7位，**最高位为0**。这样就区分开了ASCII和GB2312。这是国标码还需加上**8080H**得到机内码的原因。

4. 汉字处理流程

◆点阵字体(位图字体)把每个字分成M×N个点(16×16、24×24),每个字形都以一组二维像素信息表示。



◆存储1024个24*24点阵的汉字字形码需要的字节数是多少？ 1024*24*3

◆位图字体难以缩放，强行放大会导致字形失真，产生马赛克（mosaic）式的锯齿边缘。



第三章 3 二进制编码 (Binary Encoding)

秦磊华 汪承宁

3.1 常见编码

问题提出：

- 如何快速实现十进制数的二进制表示？
- 为什么需要提高硬件可靠性？有哪些提高可靠性的方法？



3.1 常见编码

1. 二进制编码的十进制数 (BCD)

- 概念

二进制编码的十进制数 (binary-coded decimal, BCD) ，用四位二进制数字表示一位十进制数字。

- 8421码 (以权重命名)

- 方便每一个数字 (digit) 对9求补 ($9-x$) ：2421码 (以权重命名)、余3码 (偏移量为3, 保证关于7.5对称, 即两边挖掉不用)

- 二进制里每一个数字对1求补 ($1-x$) ，相当于按位求非 (取了个巧, 把 arithmetic 化为了 logic) 。

3.1 常见编码

1. BCD编码– 8421码

- 用4位二进制数字表示一位十进制数字的一种**有权码**，4位二进制数字从高位至低位的权依次为8、4、2、1。
- 8421码编码的0 ~ 9与用4位二进制数表示的0 ~ 9相同，**减少了二进制数与十进制数之间转换的环节**，故是一种人机交互时广泛使用的中间形式。
- 不允许出现**1010 ~ 1111**六种组合(因为没有十进制数字与其对应)。
- 十进制数字0 ~ 9的8421码与相应ASCII码的低四位相同，有利于简化BCD码与字符代码的转换。



3.1 常见编码

ASCII 字符代码表 一

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符									
		0000					0001					0010	0011	0100	0101	0110	0111				
		0					1					2	3	4	5	6	7				
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符
0000	0	0	BLANK NULL	^@	NUL 空	16	▶	^P	DLE 数据链路转意	32		48	0	64	@	80	P	96	`	112	p
0001	1	1	☺	^A	SOH 头标开始	17	◀	^Q	DC1 设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q
0010	2	2	☹	^B	STX 正文开始	18	↕	^R	DC2 设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r
0011	3	3	♥	^C	ETX 正文结束	19	!!	^S	DC3 设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s
0100	4	4	♦	^D	EOT 传输结束	20	¶	^T	DC4 设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t
0101	5	5	♣	^E	ENQ 查询	21	♫	^U	NAK 反确认	37	%	53	5	69	E	85	U	101	e	117	u
0110	6	6	♠	^F	ACK 确认	22	■	^V	SYN 同步空闲	38	&	54	6	70	F	86	V	102	f	118	v
0111	7	7	●	^G	BEL 震铃	23	↕	^W	ETB 传输块结束	39	'	55	7	71	G	87	w	103	g	119	w
1000	8	8	◼	^H	BS 退格	24	↑	^X	CAN 取消	40	(	56	8	72	H	88	X	104	h	120	x
1001	9	9	○	^I	TAB 水平制表符	25	↓	^Y	EM 媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y
1010	A	10	◻	^J	LF 换行/新行	26	→	^Z	SUB 替换	42	*	58	:	74	J	90	Z	106	j	122	z
1011	B	11	♂	^K	VT 垂直制表符	27	←	^[	ESC 转意	43	+	59	;	75	K	91	[	107	k	123	{
1100	C	12	♀	^L	FF 换页/新页	28	└	^\	FS 文件分隔符	44	,	60	<	76	L	92	\	108	l	124	
1101	D	13	🎵	^M	CR 回车	29	↔	^]	GS 组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}
1110	E	14	🎶	^N	SO 移出	30	▲	^6	RS 记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~
1111	F	15	🎷	^O	SI 移入	31	▼	^-	US 单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ "Back Space"

注：表中的ASCII字符可以用:ALT + “小键盘上的数字键”输入

3.1 常见编码

1. BCD编码– 8421码

例1 8421码与十进制数之间的互换

- 按位进行，即十进制数的每一位与4位二进制编码对应。

$$(258)_{10} = (0010\ 0101\ 1000)_{8421\text{码}}$$

新问题: 两个数的BCD 码能直接实现加法运算？(16+18)

- BCD加法：节省了十一二转换时间，有利于提高运算器速度

BCD码加(减)运算修正的条件和方法是：

- ①低4位向高4位进(借)位，低4位加(减)6修正；
- ②低4位出现非法码，低4位加(减)6修正；
- ③高4位出现非法码，高4位加(减)6修正；
- ④高4位向更高位进(借)位，高4位加(减)6修正；



3.1 常见编码

1. BCD编码– 2421码

- 用4位二进制位表示一位十进制数字的一种**有权码**，4位二进制位从高至低的权依次为2、4、2、1。
- 一个十进制数字X的2421码为 $a_4a_3a_2a_1$ ，则对应的十进制的值为：
$$X = 2a_4 + 4a_3 + 2a_2 + a_1$$
- 不具备单值性：0101、1011 都对应十进制数 5。
- 为保持与十进制数的唯一对应，2421码不使用0101 ~ 1010这6种编码。
- 2421 BCD码具有自补性，是一种“对9的自补码”：2421码按位取反，便可得到该数对9的补数的2421码。

3.1 常见编码

1. BCD编码– 2421码

不使用的连续6种编码

0101
0110
0111
1000
1001
1010

先确定0到4（一半的数字），后面的再按位取反

把中间的编码挖掉不用

十进制数字	2421码
0	0000
1	0001
2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

- 两个2421码相加时，若出现最高位进位的伪码组，则减0110调整（16+16）
- 两个2421码相加时，若出现伪码组但未出现最高位进位，则加0110调整



3.1 常见编码

1. BCD编码– 余3码

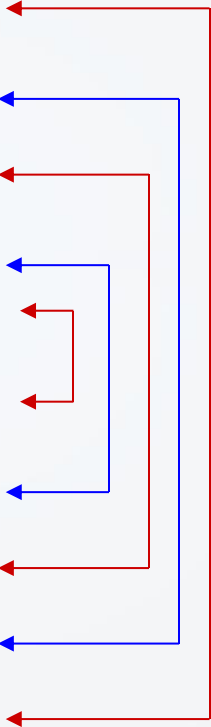
- 在8421 BCD码的基础上，每位十进制数字再加上二进制数0011得到;
- 余3码中无0000 ~ 0010、1101 ~ 1111这6种代码;
- 余3码也具有自补性，也是一种“对9的自补码”。按位取反,便可得到该数对9的补数。

3.1 常见编码

1. BCD编码- 余3码

把两边挖掉不用
十进制关于9互补

十进制字符	余3码
0-0000	0011
1-0001	0100
2-0010	0101
3-0011	0110
4-0100	0111
5-0101	1000
6-0110	1001
7-0111	1010
8-1000	1011
9-1001	1100



十六进制关于15互补



3.2 可靠性编码

问题提出:

- 为什么要引入可靠性编码?
- 如何编码才能保障可靠性?



3.2 可靠性编码

1.引入可靠性编码的原因

- 自然界存在多种干扰;
- 二进制数据在被处理、传输、存储的过程中,受干扰易发生错误;

2.为什么要引入可靠性检测?

- 对于分步骤、分阶段的计算,编码与数据在同一层,便于在微架构层面检错与纠错,可保证每一步处理的正确性,为ISA提供正确的语义支撑。



3.2 可靠性编码

3.格雷码(Gray code)

- 从普通二进制码到格雷码的编码规则

设二进制码为 $B = B_{n-1}B_{n-2} \cdots B_{i+1}B_i \cdots B_1B_0$, 对应格雷码为 $G = G_{n-1}G_{n-2} \cdots G_{i+1}G_i \cdots G_1G_0$

$$G_{n-1} = B_{n-1}$$

$$G_i = B_{i+1} \oplus B_i \quad 0 \leq i \leq n-2$$

其中, “ $\oplus$ ”为“异或” (exclusive or, XOR) 操作符 (operator) 。

3.2 可靠性编码

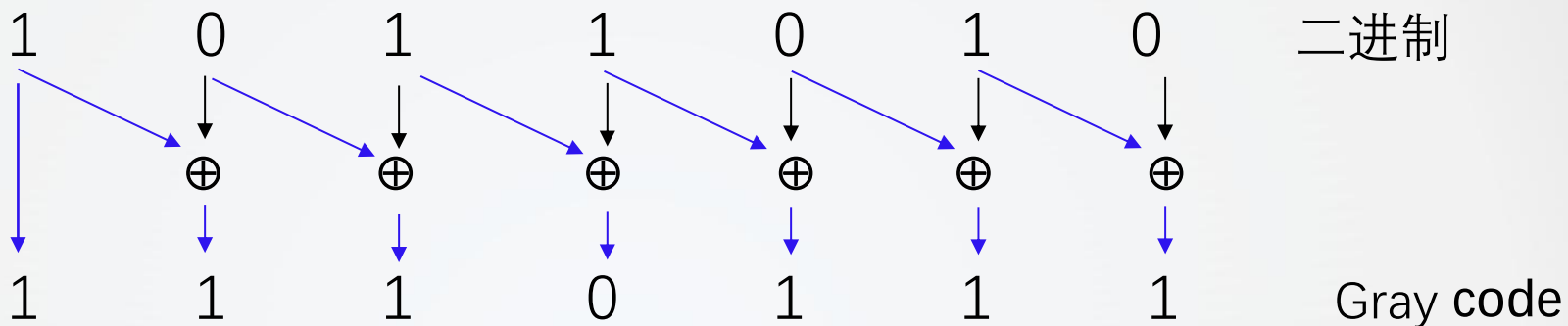
3.格雷(Gray)码

Encoding

$$G_{n-1} = B_{n-1}$$

$$G_i = B_{i+1} \oplus B_i$$

在等式两边的左侧
插入 $B_{i+1} \oplus ()$ ，可
得下式

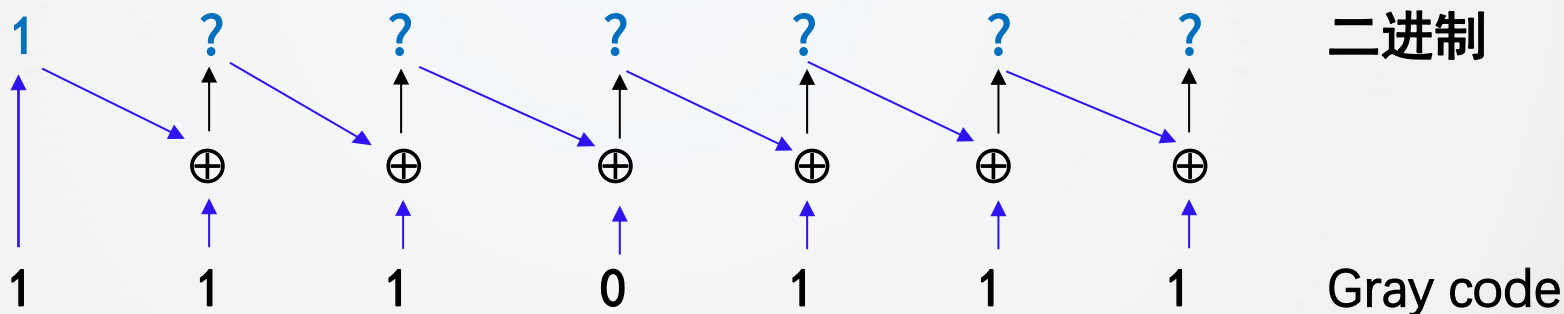


对于编码，结果的相邻比特之间没有数据依赖（data dependence），可以实现比特级并行性（bit-level parallelism）！

Decoding

$$B_{n-1} = G_{n-1}$$

$$B_i = B_{i+1} \oplus G_i$$



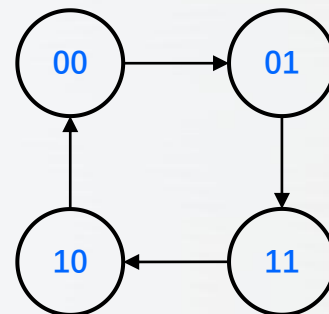
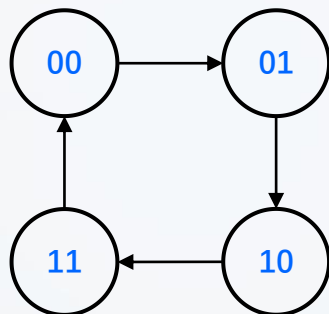
对于解码，结果的相邻比特之间有数据依赖（要用到还没有算出来的数据），不能实现比特级并行性！（除非变换计算方法）

3.2 可靠性编码

3.格雷(Gray)码

十进制数	二进制数	典型格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

格雷码**首尾相连**（ring），任意两个相邻数仅有一位不同！



哪种编码的模4计数器更可靠？格雷码



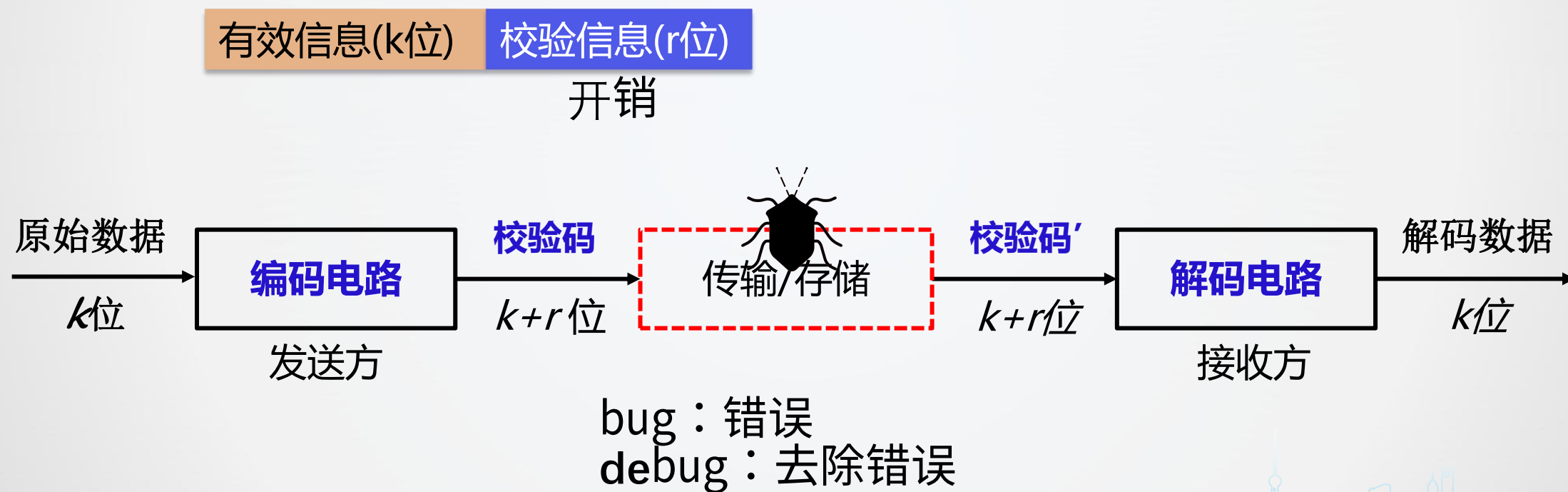
1. 为什么要引入校验码

- ◆ 信息在被处理、传输、存储的过程中，受外界环境干扰或器件本身随机涨落的影响，可能发生错误；
- ◆ **权衡 (trade-offs) 原理**：若能在微架构层面，设计检错和纠错机制，可减少上层软件检错的性能代价(**hardware-software trade-offs**)。

3.3 校验码

2. 校验码的基本原理

- ◆ 增加冗余位（校验位），即增大存储开销和计算开销（相比没有编码方案），用存储和计算来换**可靠性**（trade-off）



3.3 校验码

3. 校验能力与编码的关系

重要思想：把代数问题变成（non-Euclidean）几何（geometry）问题！

1) 汉明距离（Hamming distance）

构造汉明空间、汉明距离：相减求绝对值再相加

◆最小汉明距离：每一位是一个维度，任意两个二进制合法编码之间不同位数的最小值

◆ 0011与0001 的码距为1，一位错误时无法识别；

◆ 0000、0011、0101、0110、1001、1010、1100、1111等编码码距为2（隔得足够开）。任何一位发生改变，如0000变成1000就跳出了有效编码集合，变成了无效编码。能够检测到这种错误。

◆编码看成n维空间中的一个点的坐标。增加冗余项（也增加了空间的维数）的目的，就是为了扩大任意两个正确编码之间的最小汉明距离。

编码优化与高维（24维、36维等）空间最密（超球）堆积有着内在、深刻的联系！[2022年Fields奖]



3.3 校验码

3. 校验能力与编码的关系

2)最小汉明距离与检错（知道一共有几个错了）、纠错（知道其中部分错误的位置）能力的关系

最小汉明距离	检错(d)	纠错(c)
1	0	0
2	1	0
3	2	1
4	3	1
5	4	2
6	5	2
7	6	3

◆最小汉明距离 $\geq d+1$:

可检测 d 个错误

◆最小汉明距离 $\geq 2c+1$:

可纠正 c 个错误

◆最小汉明距离 $\geq d+c+1$:

可检测 d 个错误，可纠正其中的 c 个错误($d > c$)

码字集合的最小汉明距离越大，检错、纠错能力越强，
但存储开销和计算开销（overhead）越大

典型：single-error correctable, double-error detectable (SEC-DED) **Hamming** error-correction code (ECC)

3.3 校验码

4. 奇偶校验 Parity Check

1) 增加冗余校验1位 (管理数据的数据, 额外存储和计算开销, 换得可靠性)

有效信息(k位) 校验位P(1位)

2) 编码: 根据有效信息计算校验信息位, 使校验码 (数据 + 1位校验信息) 中1的个数满足奇数或偶数的要求

$0001 \rightarrow 0001\mathbf{1}$ (偶校验, 1的个数总数为偶数)

$0001 \rightarrow 0001\mathbf{0}$ (奇校验, 1的个数总数为奇数)

奇校验: $P = \overline{D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8}$

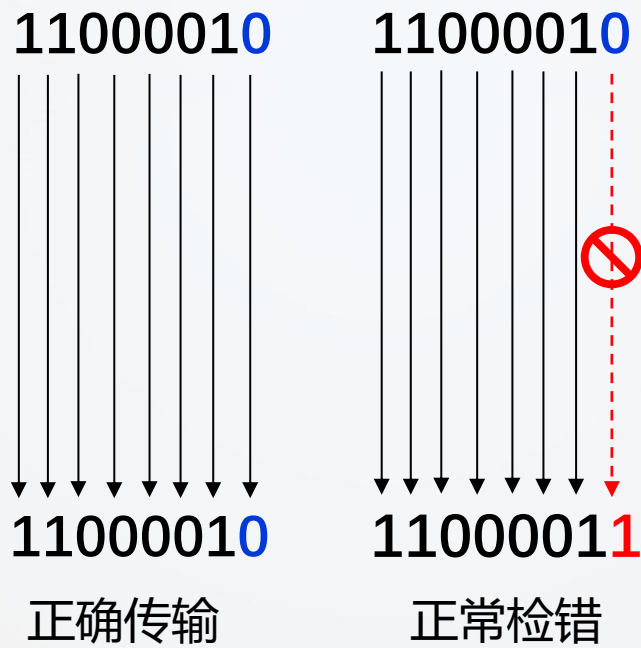
偶校验: $P = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8$ (更常用)

3.3 校验码

3) 检错方法

奇校验检错
$$G = P \oplus D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7$$

假设最多只可能发生1比特错误，G=0表示数据正常，否则表示出错。



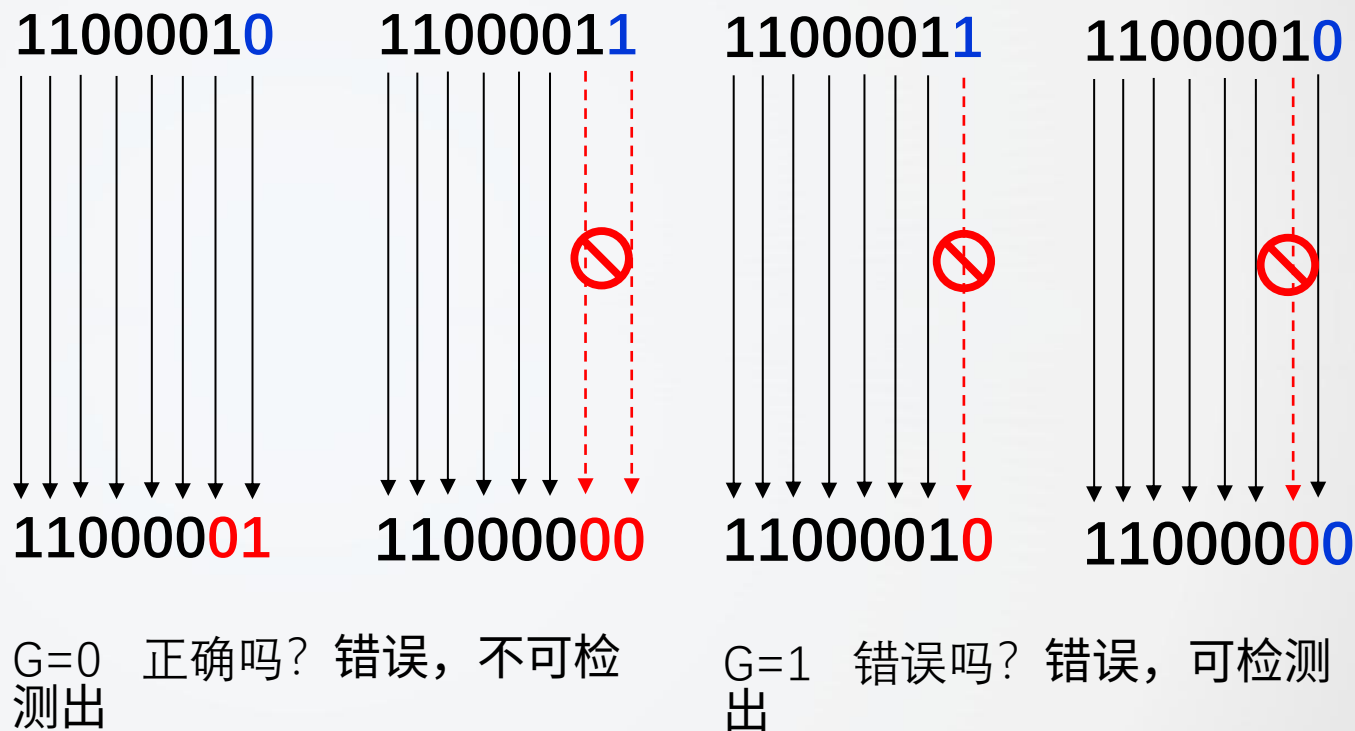
3.3 校验码

4) 奇偶校验的特点

- ◆ 编码与检错简单
- ◆ 不能检测出同时发生偶数个错误的情况,即无错结论不可信。
- ◆ 不能定位发生错误的位置,即不能纠错。

传输过程中数据位和校验位均有可能发生错误。

发生2比特错误 发生2比特错误



3.3 校验码

5) 改进的奇偶校验

行列奇偶校验
(Row and Column Parity)
也称垂直水平校验

0110100	1
1011010	0
0010110	1
1110101	1
1001011	0
1000110	1

0110100	1	
1011010	0	
0010111	1	▲
1110101	1	
1001011	0	
1000110	1	
(可纠错) ▲		
0110100	1	
1011010	0	
0110110	1	▲
1110101	1	
1001001	0	▲
1000110	1	
(可检错) ▲ ▲		

思想：用行和列共同定位错误的位置

- ◆可定位1位错的位置，并纠正（翻转）
- ◆可检测出某行(列)上的奇数位错;
- ◆可检出一部分偶数位错
- ◆不能检测出错位分布在矩形4个顶点上的错误

0110100	1
1011010	0
0110100	1
1110101	1
1001011	0
1000110	1
(可检错) ▲ ▲	

1110101	1
1011010	0
0010110	1
1110101	1
1001011	0
0000111	1
(不能检错)	

3.3 校验码

6) 奇偶校验应用场景

◆ 哪些场合应用奇偶校验?



哪根内存条最有可能具有SEC-DED Hamming ECC功能?

◆ 工程上的应用

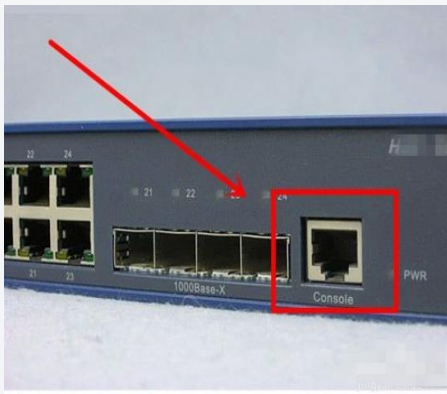
<http://www.eepw.com.cn/article/280413.htm> (关于串口奇偶校验配置的经验)

一般在同步传输方式中常采用奇校验，异步传输方式中常采用偶校验

0000000	00000000	偶校验
	00000001	奇校验

3.3 校验码

6) 奇偶校验应用场景



3.3 校验码

5. CRC校验 (循环冗余校验, cyclic redundancy check)

1) 增加冗余码 (校验位)

有效信息(k位)

校验信息(r位)

$$N=k+r \leq 2^r - 1$$

2) 生成多项式G(x) generation polynomial

- ◆ 计算校验信息所需要的除数;
- ◆ 最高位和最低位均为1;
- ◆ 任意1位发生错误都应使余数不为0;
- ◆ 不同位发生错误余数不同;
- ◆ 余数左移一位继续使用生成多项式作模2除, 应使余数循环。

3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

3) 模2运算法则

◆加法/减法：按位加不进位、减不借位，即按位（bit-wise）异或 $\oplus$ 运算

$$0 \pm 0 = 0, \quad 0 \pm 1 = 1, \quad 1 \pm 0 = 1, \quad 1 \pm 1 = 0$$

$$1101 \pm 1110 = 0011 \quad 1101 \oplus 1110 = 0011$$

◆除法：按模2减，求部分余数，不借位

3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

3) 模2运算法则

◆ 上商原则

- ① 余数首位为1时，商为1，按模2减法运算减除数；
- ② 余数首位为0时，商为0，**减0**；
- ③ 当余数位数小于除数位数时，该余数即为最后余数。

$$\begin{array}{r} 1110 \leftarrow \text{商} \\ 1011 \overline{) 1100000} \\ \underline{1011} \\ 1110 \\ \underline{1011} \\ 1010 \\ \underline{1011} \\ 0010 \\ 0000 \\ \hline 010 \text{ 余数} \end{array}$$

3.3 校验码

5. CRC校验 (Cyclic Redundancy Check)

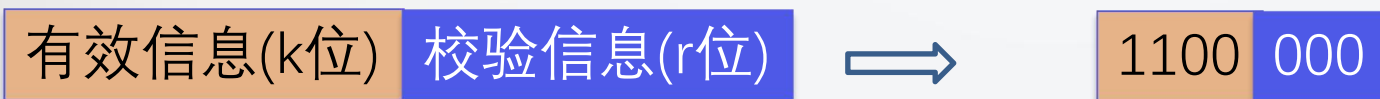
4) CRC 编码方法

(1)根据待校验信息的长度 k , 按照 $k+r \leq 2^r - 1$ 确定校验位 r 的位数

如对4位信息 1100 进行CRC编码, 根据 $4+r \leq 2^r - 1$, 得 $r_{\min} = 3$

(2)根据 r 和生成多项式 G 的关系, 任选位数为 $r+1$ 的生成多项式 $G(X) = 1011$

(3)对原始信息进行下列变化



即: 将待校验的二进制信息 K 逻辑左移 r 位, 得到 K'

3.3 校验码

(4)对K'按模2运算法则除以G(x)，求CRC编码中的r位校验信息

$$\begin{array}{r} 1110 \\ 1011 \overline{) 1100000} \\ \underline{1011} \\ 1110 \\ \underline{1011} \\ 1010 \\ \underline{1011} \\ 0010 \\ \underline{0000} \\ 010 \end{array}$$

(5)用得到的余数替换K'的最后r位全0，即可得到1110对应的CRC编码



3.3 校验码

5) CRC检错

接收方利用同样的G(X)，对收到的编码做模2除运算

$$\begin{array}{r}
 1110 \\
 1011 \overline{) 1100010} \\
 \underline{1011} \\
 1110 \\
 \underline{1011} \\
 1011 \\
 \underline{1011} \\
 0000
 \end{array}$$

余数为全0说明传输没有错误，否则说明有错。

3.3 校验码

5) CRC检错

$$\begin{array}{r} 1110 \\ 1011 \overline{) 1100\color{red}110} \end{array}$$

$$\begin{array}{r} 1111 \\ 1011 \overline{) 110\color{red}1010} \\ \underline{1011} \\ 1100 \\ \underline{1011} \\ 1111 \\ \underline{1011} \\ 1000 \\ \underline{1011} \\ 011 \end{array}$$

$$\begin{array}{r} 1111 \\ 1011 \overline{) 110\color{red}110} \\ \underline{1011} \\ 110\color{red}1 \\ \underline{1011} \\ 1101 \\ \underline{1011} \\ 1100 \\ \underline{1011} \\ 111 \end{array}$$

余数不为0说明传输有错

3.3 校验码

5) CRC检错

7+4编码的错误位置与余数的函数关系

A ₇ ~A ₁	余数	出错位
1100010	000	无
1100011	001	1
1100000	010	2
1100110	100	3
1101010	011	4
1110010	110	5
1000010	111	6
0100010	101	7

$G(X) = 1011$

A ₇ ~A ₁	余数	出错位
1100101	000	无
1100100	001	1
1100111	010	2
1100001	100	3
1101101	101	4
1110101	111	5
1000101	011	6
0100101	110	7

$G(X) = 1101$

注意：下标从1开始编号

同一位出错的情况下，CRC校验得到的余数与G(X)的取值有关；

如果有且只有1位出错，能根据余数定位错误的位置



3.3 校验码

5) CRC检错

A ₇ ~A ₁	余数	出错位
1100010	000	无
1100011	001	1
1100000	010	2
1100110	100	3
1101010	011	4
1110010	110	5
1000010	111	6
0100010	101	7

有2位同时发生错误

1011

1111

110110

1011

1101

1011

1101

1011

111

- ◆ 如果有2位同时发生错误，CRC能判断出出错了（detectable），但是不能正确定位错误的位置（uncorrectable），即不知道错在哪。
- ◆ CRC能检出3位及以上错误吗？

位置判断不对



3.3 校验码

5) CRC检错

A ₇ ~A ₁	余数	出错位
1100101	000	无
1100100	001	1
1100111	010	2
1100001	100	3
1101101	101	4
1110101	111	5
1000101	011	6
0100101	110	7

$G(X) = 1101$

3位都出错

◆ 1100 101 -----> 1101 0 0 0

↓

校验结果余数为0 误认为没错

◆ 1100 101 -----> 1101 0 1 0

↓

校验结果余数为010 可判断出现了错误，但位置不对

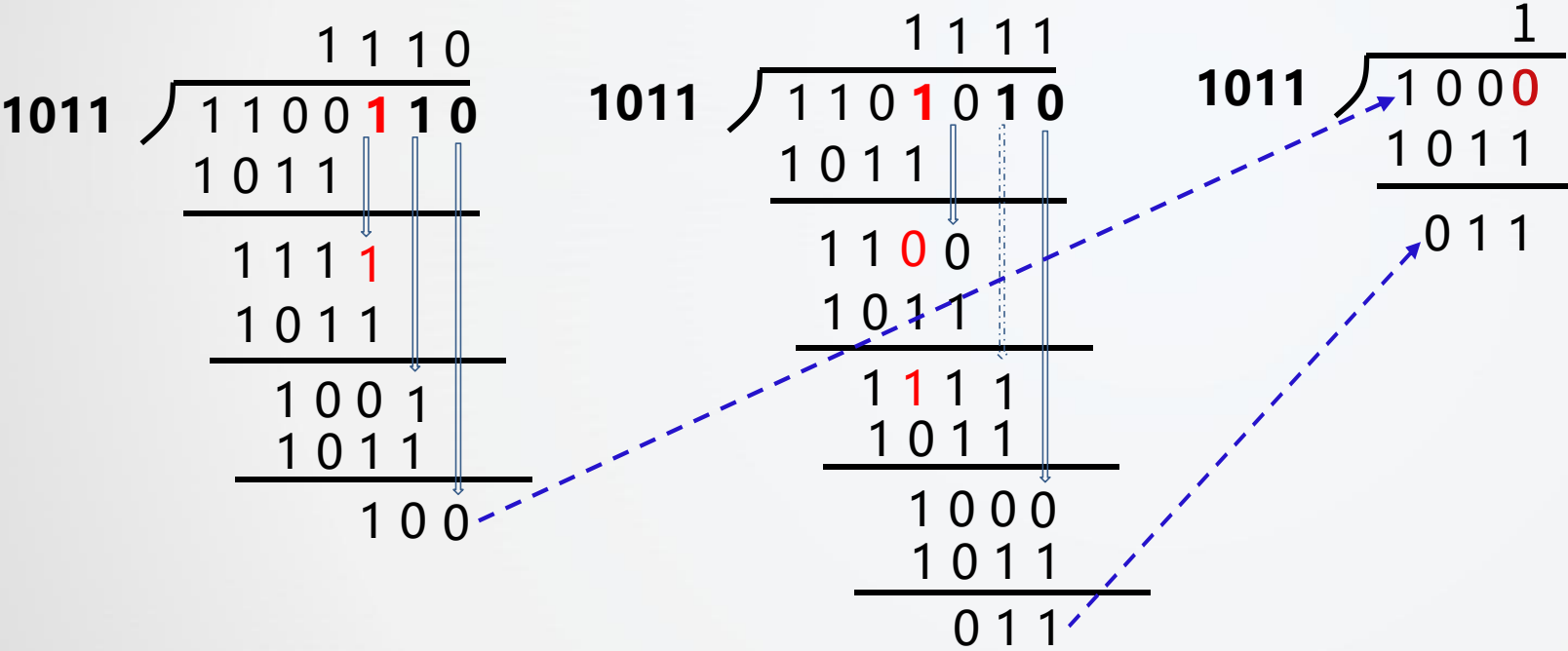
CRC不能检出所有3位同时出错的情况!
即不能检出3位同时出错（不可靠）。



3.3 校验码

5) CRC检错

有且只有一位出错情况下，余数的循环（cyclic）特性



A ₇ ~A ₁	余数	出错的位
1100010	000	无
1100011	001	1
1100000	010	2
1100110	100	3
1101010	011	4
1110010	110	5
1000010	111	6
0100010	101	7

$G(X)=1011$

3.3 校验码

6) CRC纠错

$G(X) = 1101$

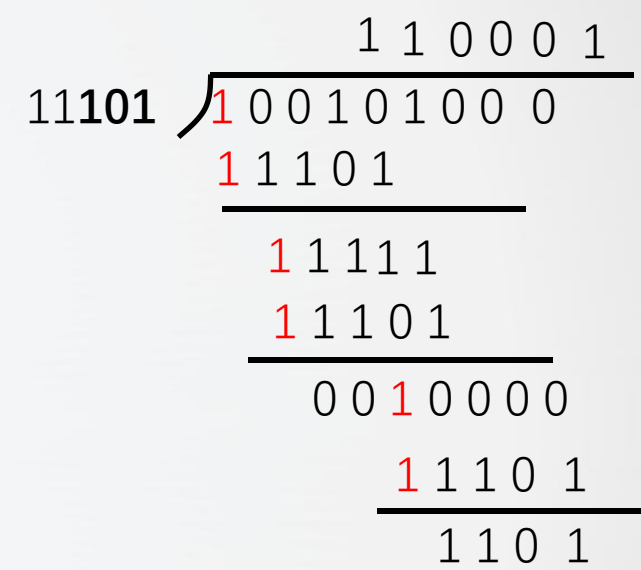
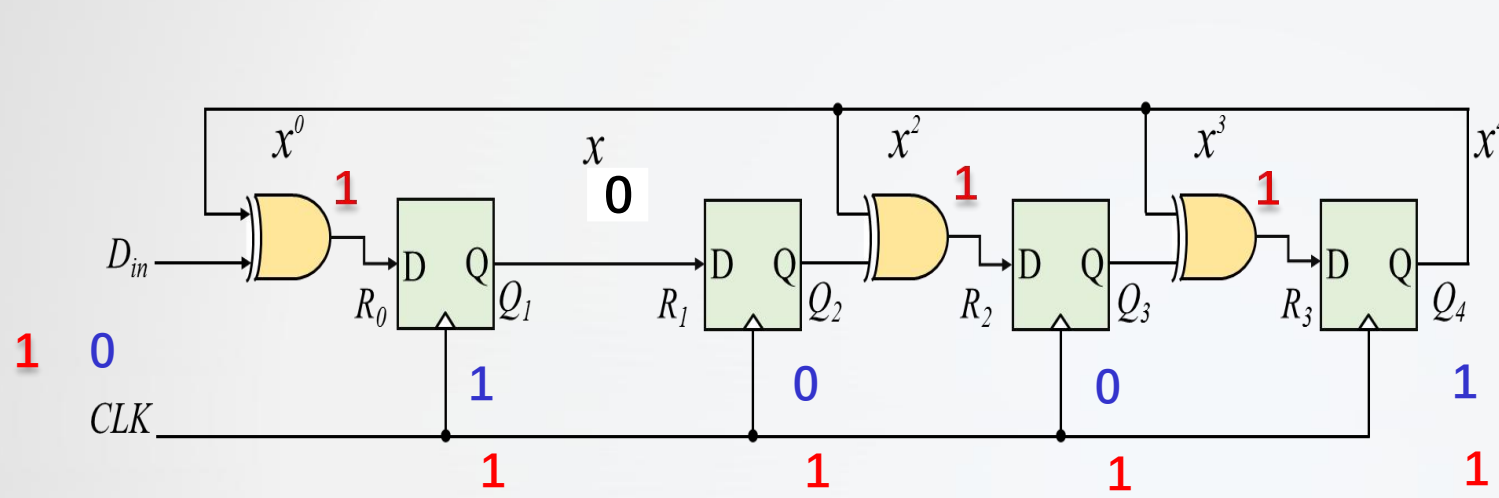
$A_7 \sim A_1$	余数	出错位
1100101	000	无
1100100	001	1
1100111	010	2
1100001	100	3
1101101	101	4
1110101	111	5
1000101	011	6
0100101	110	7

100 { 100 0/1101 → 101 0/1101 → 111 0/1101 → 011 0/1101 → 110 0/1101
1100001 循环左移 → 1000011 循环左移 → 0000111 循环左移 → 0001110 循环左移 → 0011100 取非 → 1011100

110 1/1101 → 001 1/1101 → 010 1/1101 → 100 1/1101
1011100 循环左移 → 0111001 循环左移 → 1110010 循环左移 → 1100101

3.3 校验码

7) CRC编码电路(串行)



- ◆ 了解 CRC串行编码电路的原理和结构
- ◆ 有异或门的位置是生成多项式中1对应的位置, $G(X)=X^4+X^3+X^2+1$ 11101
- ◆ D触发器初始状态为0
- ◆ 最后余数从高到低分别为 $Q_4 \sim Q_1$

3.3 校验码

7) CRC编码电路(并行)

◆ 两数的模的异或等于两数异或后的模 (modulo)

$$(M(x) \% G(x)) \oplus (N(x) \% G(x)) = (M(x) \oplus N(x)) \% G(x)$$

◆ (7,4) CRC编码, 生成多项式 $G(x)=1011$, 原始数据 $M(x)=1101$, 求CRC编码

1101	<u>000</u>	<u>%G(x)</u>	----->	001
=	(1000	<u>000</u>	<u>%G(x)</u>)	-----> 101
				⊕
⊕	(0100	<u>000</u>	<u>%G(x)</u>)	-----> 111
				⊕
⊕	(0001	<u>000</u>	<u>%G(x)</u>)	-----> 011

◆ 先算(查) 2^6 、 2^5 、 2^4 、 2^3 等常量位的余数, 再将其组合求解任意编码的余数

3.3 校验码

8) 软件实现方式

/******该文件使用查表法计算CCITT 标准的CRC-16检验码，并附测试代码******/

```
#include
```

```
#define CRC_INIT 0xffff //CCITT初始CRC为全1
```

```
#define GOOD_CRC 0xf0b8 //校验时计算出的固定结果值
```

/****下表是常用ccitt 16,生成式OX1021反转成OX8408后的查询表格*****/ (注意，低位先传送时，生成多项式应反转(低位与高位互换))。

```
unsigned short crc16_ccitt_table[256] =
```

```
{ 0x0000, 0x1189, 0x2312, 0x329b, 0x4624, 0x57ad, 0x6536, 0x74bf, 0x8c48, 0x9dc1, 0xaf5a, 0xbed3, 0xca6c, 0xdbe5, 0xe97e, 0xf8f7,
0x1081, 0x0108, 0x3393, 0x221a, 0x56a5, 0x472c, 0x75b7, 0x643e, 0x9cc9, 0x8d40, 0xbfdb, 0xae52, 0xdaed, 0xcb64, 0xf9ff, 0xe876, 0x2102,
0x308b, 0x0210, 0x1399, 0x6726, 0x76af, 0x4434, 0x55bd, 0xad4a, 0xbcc3, 0x8e58, 0x9fd1, 0xeb6e, 0xfae7, 0xc87c, 0xd9f5, 0x3183, 0x200a,
0x1291, 0x0318, 0x77a7, 0x662e, 0x54b5, 0x453c, 0xbdcb, 0xac42, 0x9ed9, 0x8f50, 0xfbef, 0xea66, 0xd8fd, 0xc974, 0x4204, 0x538d, 0x6116,
0x709f, 0x0420, 0x15a9, 0x2732, 0x36bb, 0xce4c, 0xdfc5, 0xed5e, 0xfcd7, 0x8868, 0x99e1, 0xab7a, 0xbaf3, 0x5285, 0x430c, 0x7197, 0x601e,
0x14a1, 0x0528, 0x37b3, 0x263a, 0xdec d, 0xcf44, 0xfddf, 0xec56, 0x98e9, 0x8960, 0xbbfb, 0xaa72, 0x6306, 0x728f, 0x4014, 0x519d, 0x2522,
0x34ab, 0x0630, 0x17b9, 0xef4e, 0xfec7, 0xcc5c, 0xdd d5, 0xa96a, 0xb8e3, 0x8a78, 0x9bf1, 0x7387, 0x620e, 0x5095, 0x411c, 0x35a3, 0x242a,
0x16b1, 0x0738, 0xffcf, 0xee46, 0xdcdd, 0xcd54, 0xb9eb, 0xa862, 0x9af9, 0x8b70, 0x8408, 0x9581, 0xa71a, 0xb693, 0xc22c, 0xd3a5, 0xe13e,
0xf0b7, 0x0840, 0x19c9, 0x2b52, 0x3adb, 0x4e64, 0x5fed, 0x6d76, 0x7cff, 0x9489, 0x8500, 0xb79b, 0xa612, 0xd2ad, 0xc324, 0xf1bf,
0xe036, 0x18c1, 0x0948, 0x3bd3, 0x2a5a, 0x5ee5, 0x4f6c, 0x7df7, 0x6c7e, 0xa50a, 0xb483, 0x8618, 0x9791, 0xe32e, 0xf2a7, 0xc03c,
0xd1b5, 0x2942, 0x38cb, 0x0a50, 0x1bd9, 0x6f66, 0x7eef, 0x4c74, 0x5dfd, 0xb58b, 0xa402, 0x9699, 0x8710, 0xf3af, 0xe226, 0xd0bd,
0xc134, 0x39c3, 0x284a, 0x1ad1, 0x0b58, 0x7fe7, 0x6e6e, 0x5cf5, 0x4d7c, 0xc60c, 0xd785, 0xe51e, 0xf497, 0x8028, 0x91a1, 0xa33a,
0xb2b3, 0x4a44, 0x5bcd, 0x6956, 0x78df, 0x0c60, 0x1de9, 0x2f72, 0x3efb, 0xd68d, 0xc704, 0xf59f, 0xe416, 0x90a9, 0x8120, 0xb3bb,
0xa232, 0x5ac5, 0x4b4c, 0x79d7, 0x685e, 0x1ce1, 0x0d68, 0x3ff3, 0x2e7a, 0xe70e, 0xf687, 0xc41c, 0xd595, 0xa12a, 0xb0a3, 0x8238,
0x93b1, 0x6b46, 0x7acf, 0x4854, 0x59dd, 0x2d62, 0x3ceb, 0x0e70, 0x1ff9, 0xf78f, 0xe606, 0xd49d, 0xc514, 0xb1ab, 0xa022, 0x92b9,
0x8330, 0x7bc7, 0x6a4e, 0x58d5, 0x495c, 0x3de3, 0x2c6a, 0x1ef1, 0x0f78 };
```

```
unsigned short do_crc(unsigned short reg_init, unsigned char *message, unsigned int len)
```

```
{ unsigned short crc_reg = reg_init;
```

```
while (len--)
```

```
    crc_reg = (crc_reg >> 8) ^ crc16_ccitt_table[(crc_reg ^ *message++) & 0xff];
```

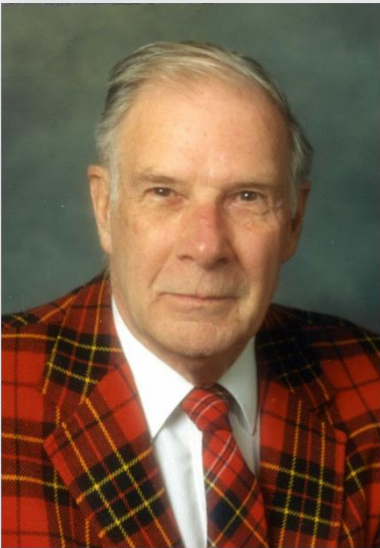
```
return crc_reg; }
```

3.3 校验码

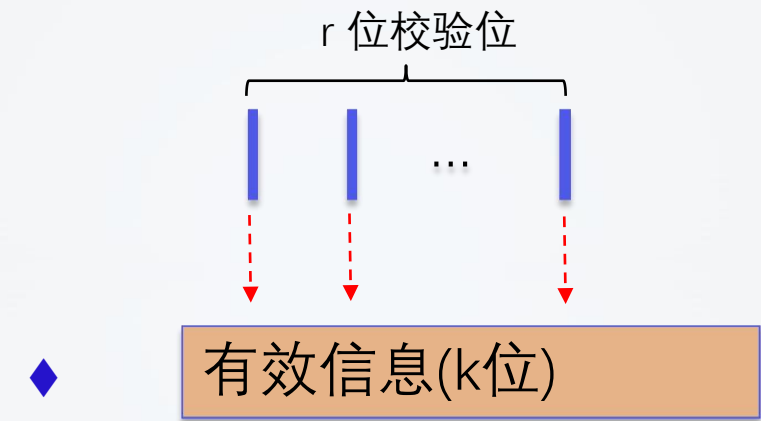
6. 海明校验 (Hamming Code, 汉明码)

1)海明校验基本概念

◆ 分组奇偶校验



Richard Wesley Hamming 1950



$$N = k + r \leq 2^r - 1$$

◆

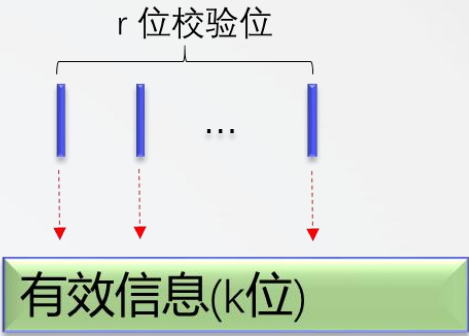
k	1	2~4	5~11	12~26	27~57	58~120	...
r	2	3	4	5	6	7	...



3.3 校验码

6. 海明校验 (Hamming Code, 汉明码)

2) 编码规则



Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

- ◆ k+r位海明码自左至右依次编为第1, 2, 3, ……., k+r位;
- ◆ r 位校验位记为 P_i ($i=1, 2, \dots, r$)分别位于海明编码的第 2^{i-1} 位, 即 H_1 、 H_2 、 H_4 、 H_8 、 $H_{16}, \dots$
- ◆ H_j 位的**数据**被编号小于j的若干个海明位号之和等于j的**校验位**所校验;



3.3 校验码

6. 海明校验

2) 编码规则

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

$P1=b1\oplus b2\oplus b4\oplus b5\oplus b7$

$P2=b1\oplus b3\oplus b4\oplus b6\oplus b7$

$P3=b2\oplus b3\oplus b4$

$P4=b5\oplus b6\oplus b7$



3.3 校验码

6. 海明校验

3) 检错

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

设置指错字 $G_4G_3G_2G_1$

$$G_4 = P4 \oplus b5 \oplus b6 \oplus b7$$

$$G_3 = P3 \oplus b2 \oplus b3 \oplus b4$$

$$G_2 = P2 \oplus b1 \oplus b3 \oplus b4 \oplus b6 \oplus b7 \quad G_1 = P1 \oplus b1 \oplus b2 \oplus b4 \oplus b5 \oplus b7$$

$G_4G_3G_2G_1$ 为0则表明无错误， 反之给出差错位的海明码位号。



3.3 校验码

6. 海明校验

4)海明校验编码与检错举例

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

设被传送的信息 $b_1b_2b_3b_4b_5b_6b_7 = 1\ 0\ 1\ 1\ 0\ 0\ 0$ ，采用偶校验；

$$\text{则： } P1=b1\oplus b2\oplus b4\oplus b5\oplus b7 = 1\oplus 0\oplus 1\oplus 0\oplus 0 = 0$$

$$P2=b1\oplus b3\oplus b4\oplus b6\oplus b7 = 1\oplus 1\oplus 1\oplus 0\oplus 0=1$$

$$P3=b2\oplus b3\oplus b4 =0\oplus 1\oplus 1 = 0$$

$$P4=b5\oplus b6\oplus b7 =0\oplus 0\oplus 0 = 0$$

得到的海明编码为 $H = 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 0$



3.3 校验码

6. 海明校验

4)海明校验编码与检错举例

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

当传输无错时，即H = 0 1 1 0 0 1 1 0 0 0 0

则： $G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$

$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$

$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$

$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$

$G_4G_3G_2G_1 = 0000$ ， 表明无错!



3.3 校验码

6. 海明校验

4)海明校验编码与检错举例

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

当传输出错时，即 $H = 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ \underline{1}$

$$\text{则: } G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$G_4 G_3 G_2 G_1 = 1\ 0\ 1\ 1, \quad \text{表明} H_{11} \text{位出错!}$$



3.3 校验码

6. 海明校验

5)海明校验特点分析

Hi	1	2	3	4	5	6	7	8	9	10	11
	1 /P1	2 /P2	1+2 /b1	4 /P3	1+4 /b2	2,4 /b3	1+2+4 /b4	8 /P4	1+8 /b5	2+8 /b6	1+2+8 /b7

(1) 指错字 $G_4G_3G_2G_1= 0000$ 一定无错吗?

当传输无错时, 即 $H = 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 0$

$$G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

能发现什么规律吗?



3.3 校验码

6. 海明校验

2) 指错字能区别一位错和多位错吗？

$$G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$b_1、b_2 \text{ 出错时: } G_4G_3G_2G_1 = 0110 \quad b_6、b_7、P_4 \text{ 出错时: } G_4G_3G_2G_1 = 1000$$

$$\text{仅 } b_3 \text{ 出错时: } G_4G_3G_2G_1 = 0110 \quad \text{仅 } P_4 \text{ 出错时: } G_4G_3G_2G_1 = 1000$$



3.3 校验码

6. 海明校验

3)指错字能区别多种不同多位同时出错的情况吗？

$$G_4 = P_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$G_3 = P_3 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$G_2 = P_2 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$G_1 = P_1 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 0$$

b_1 、 b_2 出错时 与 P_2 、 P_3 同时出错时候得到的指错字相同

总结论：在能确定只有一位出错时，出错结论可靠且能定位错误；否则，出错结论可靠但不能定位，无错结论不可靠。



3.3 校验码

7. 校验码应用

◆ 廉价磁盘的冗余阵列(RAID, David A. Patterson)

